

Fleet Competition: Adversarial Evidence Selection via Tournament Protocols

Halley Young, Microsoft Research

Paper 17 in the Judgment Geometry series March 22, 2026

Abstract

We introduce *fleet competition*, a tournament-based protocol for adversarial evidence selection in the JUDGE framework. When multiple evidence-producing strategies compete over a single judgment coordinate, greedy selection fails: a locally best strategy may be globally dominated by a peer strategy that exposes contradictions. Fleet competition replaces greedy search with a sequence of *competition rounds*, each consisting of a bid phase (strategies propose evidence), an evaluation phase (a judge scores bids), and an optional *challenge phase* in which one strategy disputes another’s bid. Disputes are resolved by an ADJUDICATOR that applies the trust algebra of the JUDGE judgment 8-tuple. Winning evidence is forwarded to the descent engine via the *mixed-evidence router*. We prove the *Fleet Quality Guarantee*: the trust level of tournament-selected evidence is at least as high as the maximum trust level achievable by any individual strategy. Experiments on 300 verification tasks show a 21% improvement in evidence trust level over greedy single-strategy selection, with per-round latency overhead of only 6.5 *ms*.

Contents

1	Introduction	3
1.1	Adversarial Selection as a Tournament	3
1.2	Contributions	3
1.3	Paper Structure	3
2	Background	4
2.1	Judgment Tuples and Trust Annotations	4
2.2	Evidence Bundles and Strategies	4
2.3	Descent Engine and Routing	4
3	The Competition Model	4
3.1	Fleet Competition	4
3.2	Competition Rounds	5
3.3	Multi-Criterion Evaluation	5
4	The Challenge Protocol	6
4.1	Challenge Outcomes	6
4.2	When to Challenge	6
4.3	Challenge Lifecycle	7
4.4	Escalation	7

5	Adjudication	7
5.1	The Adjudicator Class	7
5.2	Dispute Resolution Strategies	8
6	Evidence Routing	9
6.1	The Mixed-Evidence Router	9
6.2	Routing Strategies	9
6.3	Conflict Resolution	10
7	The Fleet Quality Guarantee	10
7.1	Statement	10
7.2	Proof	10
7.2.1	Upper Bound	11
7.2.2	Lower Bound	11
7.3	Corollaries	11
8	Experiments	12
8.1	Setup	12
8.2	Results	12
8.3	Effect of Challenge Protocol	12
8.4	Routing Efficiency	13
8.5	Ablation: Maximum Rounds	13
9	Related Work	13
10	Conclusion	14

1 Introduction

The JUGE framework represents program verification as navigation through a semantic site: a judgment $\mathcal{J} = (\mathbf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ is a structured 8-tuple carrying a trust annotation $\mathcal{T} \in \mathfrak{T}$ that encodes how much epistemic weight the framework assigns to the evidence bundle $\mathcal{E}$. Papers 1–4 established the foundations of this algebra; Papers 5–16 built an increasingly rich library of semantic moves, proof-carrying adapters, and cohomological obstructions.

A recurring bottleneck in practice is the *evidence selection problem*: given a judgment coordinate $\mathbf{c}$ and several strategies capable of producing evidence for the proposition φ , which strategy should the orchestrator invoke? The naive greedy answer—*invoke the strategy with the highest self-declared trust ceiling*—has a critical failure mode. A strategy may legitimately claim a high trust ceiling yet produce evidence that, when examined by a peer strategy, is shown to rest on contradicted assumptions. The peer’s CHALLENGE move then forces a demotion of the trust annotation, wasting computational budget and delaying verification.

1.1 Adversarial Selection as a Tournament

We argue that the correct abstraction is not selection but *competition*. A *fleet* $\mathcal{F} = \{S_1, \dots, S_n\}$ of evidence-producing strategies operates as a population of adversaries: each strategy is simultaneously a *bidder* (proposing evidence it believes is good) and a *challenger* (empowered to dispute bids from other strategies). The interplay between bidding and challenging drives the population toward evidence whose trust level survives adversarial scrutiny.

This mirrors the design of tournament-style algorithms in combinatorial optimization [1] and the challenge–response protocols of formal argument systems [2], but instantiated within the algebraic structure of the trust lattice $\mathfrak{T}$ defined in Paper 4.

1.2 Contributions

The contributions of this paper are:

- (i) The *fleet competition model*: formal definitions of FLEETCOMP, COMPROUND, and BIDOUTCOME (section 3).
- (ii) The *challenge protocol*: definitions of CHALOUT, challenge conditions, and escalation semantics (section 4).
- (iii) The *adjudication calculus*: the ADJUDICATOR class and its dispute-resolution strategies (section 5).
- (iv) The *mixed-evidence router*: forwarding winning evidence to the descent engine via EV-ROUTER and ROUTSTRAT (section 6).
- (v) The *Fleet Quality Guarantee*: a theorem showing that tournament selection cannot do worse than the best individual strategy (section 7).
- (vi) Experimental validation on 300 verification tasks (section 8).

1.3 Paper Structure

Section 2 reviews the judgment 8-tuple and trust algebra prerequisites. Section 3 through section 6 develop the four main components. Section 7 presents the quality guarantee with proof. Section 8 reports empirical results. Section 10 concludes.

2 Background

2.1 Judgment Tuples and Trust Annotations

A *JUGEO judgment* is an 8-tuple

$$\mathcal{J} = (\mathbf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$$

where $\mathbf{c}$ is a semantic coordinate (module, function, interface, or test region), φ is the proposition under verification, $\mathcal{A}$ is the carrier type, $\mathcal{E}$ is the evidence bundle, $\mathcal{O}$ is the obligation set, $\mathcal{B}$ is the obstruction class, $\mathcal{T} \in \mathfrak{T}$ is the trust annotation, and Π is the provenance record.

The trust algebra $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$ is a totally ordered bounded lattice with the chain

$$\text{CONTRADICTED} \preceq \text{UNVERIFIED} \preceq \text{COPILOT} \preceq \text{ORACLE} \preceq \text{RUNTIME} \preceq \text{SOLVER} \preceq \text{PROOF}.$$

Evidence at level τ in the chain may be *promoted* to level $\tau' \succ \tau$ by supplying a proof obligation π , and *demoted* to level $\tau'' \prec \tau$ by a valid counter-evidence certificate χ . Full details appear in Paper 4.

2.2 Evidence Bundles and Strategies

An *evidence strategy* S is a function $S : (\mathbf{c}, \varphi, \mathcal{A}) \rightarrow \mathcal{E} \times \mathcal{T}$ that produces an evidence bundle together with a trust annotation for that bundle. Each strategy has a declared *trust ceiling* $\tau^*(S) \in \mathfrak{T}$: the highest trust level it can claim for its outputs.

Strategies may be *solver-based* (discharging SMT obligations in `QF_LIA` or `QF_LRA`), *runtime-witnessed* (accumulating test-case verdicts), *proof-backed* (invoking a `LEAN4` checker), or *copilot-suggested* (language-model inference with no formal backing). The fleet competition protocol is strategy-agnostic: it treats all strategies uniformly as bid producers.

2.3 Descent Engine and Routing

The *descent engine* (Papers 3, 11, 13) propagates evidence from a covering family $\{U_i\}$ of a semantic site S to the global section over $\bigcup U_i$. A descent step consumes an evidence bundle and a gluing certificate, and elevates the trust annotation when the Čech boundary condition $\delta(\mathcal{E}) = 0$ is met. Routing determines which channel of the descent engine receives the winning evidence after each competition round.

3 The Competition Model

3.1 Fleet Competition

Definition 3.1 (`FLEETCOMP`). A *fleet competition* is a tuple $\text{FLEETCOMP} = (\mathcal{F}, \mathbf{c}, \varphi, \text{Eval}, r_{\max})$ where:

- $\mathcal{F} = \{S_1, \dots, S_n\}$ is a non-empty fleet of evidence strategies,
- $(\mathbf{c}, \varphi)$ is the judgment coordinate and proposition under competition,
- $\text{Eval} : \mathcal{B} \rightarrow \mathbb{R}_{>0}$ is a judge function scoring bids,
- $r_{\max} \in \mathbb{N}_{>0}$ is the maximum number of rounds.

The competition proceeds in rounds. At round $r \in \{1, \dots, r_{\max}\}$ all strategies still active in the tournament simultaneously submit bids. The evaluator selects winners. Eliminated strategies are deactivated; surviving strategies advance to the next round. The competition terminates when either (a) exactly one strategy remains, or (b) round $r_{\max}$ is reached, in which case the strategy with the highest cumulative bid score is declared the winner.

Definition 3.2 (Bid). A *bid* submitted by strategy S_i in round r is a tuple

$$B_i^{(r)} = (S_i, \mathcal{E}_i, \mathcal{T}_i, \mathbf{q}_i, u_i)$$

where $\mathcal{E}_i$ is the proposed evidence bundle, $\mathcal{T}_i \in \mathcal{T}$ is the claimed trust level, $\mathbf{q}_i \in [0, 1]$ is a self-scored semantic quality in the trust algebra, and $u_i \in [0, 1]$ is the epistemic uncertainty.

Definition 3.3 (BIDOUTCOME). A *bid outcome* $\text{BIDOUTCOME} \in \{\text{ACCEPT}, \text{REJECT}, \text{CHALLENGE}, \text{EXPIRED}\}$ records the final disposition of a bid:

- **ACCEPT**: the bid was selected as a round winner.
- **REJECT**: the bid was not selected; the round closed without it.
- **CHALLENGE**: the bid was challenged by a peer strategy.
- **EXPIRED**: the round closed before the bid was evaluated.

3.2 Competition Rounds

Definition 3.4 (COMROUND). A *competition round* $R^{(r)}$ is a tuple $(\mathcal{B}^{(r)}, \text{phase}, \text{win}^{(r)}, \text{lose}^{(r)}, \text{challenges}^{(r)})$ where $\mathcal{B}^{(r)}$ is the set of bids submitted in round r , $\text{phase} \in \{\text{OPEN}, \text{EVALUATING}, \text{CLOSED}, \text{ARCHIVED}\}$ is the lifecycle phase, $\text{win}^{(r)} \subseteq \mathcal{F}$ and $\text{lose}^{(r)} \subseteq \mathcal{F}$ are the winning and losing strategy sets, and $\text{challenges}^{(r)}$ is the (possibly empty) list of challenge records issued during round r .

Rounds progress through four phases that enforce ordering constraints on state transitions. The **OPEN** phase accepts new bids. Once bid collection closes the round enters **EVALUATING**, during which the judge and challenge protocol run. The round becomes **CLOSED** once a winner is determined, and **ARCHIVED** when the round's data is persisted.

3.3 Multi-Criterion Evaluation

The evaluator function **Eval** combines three criteria with configurable weights $w_v, w_s, w_u \geq 0$ summing to 1:

$$\text{Eval}(B_i) = w_v \cdot \text{val}(B_i) + w_s \cdot \mathbf{q}_i - w_u \cdot u_i,$$

where $\text{val}(B_i)$ is a strategy-specific utility score provided by the strategy at bid time. Before elimination, the evaluator applies a *Pareto filter*: bids that are strictly dominated on all three axes (lower value, lower quality, higher uncertainty) are pruned regardless of their composite score.

Example 3.5 (Solver vs. runtime strategy). Let S_1 be a **QF_LIA** solver strategy with trust ceiling **SOLVER** and S_2 be a runtime-witness strategy with ceiling **RUNTIME**. In round 1, S_1 submits $B_1 = (S_1, \mathcal{E}_{\text{unsat}}, \text{SOLVER}, 0.95, 0.03)$ and S_2 submits $B_2 = (S_2, \mathcal{E}_{42}, \text{RUNTIME}, 0.78, 0.12)$. The Pareto filter retains both (neither dominates the other on all three axes). **Eval** with $w_v = 0.4, w_s = 0.4, w_u = 0.2$ scores S_1 at 0.817 and S_2 at 0.588. S_1 advances; S_2 is eliminated with **BIDOUTCOME** = **REJECT**.

The following Python listing shows the core bid-collection and evaluation loop as implemented in `fleet_competition/algorithms.py`:

Listing 1: Multi-criterion bid evaluation loop (`fleet_competition/algorithms.py`)

```

1 def evaluate_round(round_: FleetRound,
2                   weights: tuple[float, float, float]) -> list[str]:
3     w_v, w_s, w_u = weights
4     active = [b for b in round_.bids if b.status == BidStatus.PENDING]
5     # Pareto filter: remove strictly dominated bids
6     frontier = pareto_filter(active)
7     # Composite score
8     scored = [(b, w_v * b.bid_value + w_s * b.semantic_score
9               - w_u * b.uncertainty)
10              for b in frontier]
11     scored.sort(key=lambda p: p[1], reverse=True)
12     winners = [b.bidder_id for b, _ in scored[:round_.max_winners]]
13     for b, _ in scored[round_.max_winners:]:
14         b.status = BidStatus.REJECTED # BIDOUTCOME ← REJECT
15     return winners

```

4 The Challenge Protocol

4.1 Challenge Outcomes

The CHALLENGE move allows a strategy S_j to dispute a bid $B_i^{(r)}$ submitted by a peer S_i . A challenge asserts that the evidence $\mathcal{E}_i$ either contains a contradiction, overstates its trust level, or is inconsistent with the challenger's own evidence $\mathcal{E}_j$.

Definition 4.1 (CHALOUT). A *challenge outcome* $\text{CHALOUT} \in \{\text{SUSTAIN}, \text{OVERTURN}, \text{ESCALATE}, \text{WITHDRAWN}\}$ records the resolution of a challenge:

- **SUSTAIN**: the challenge is upheld; bid B_i is overturned with $\text{BIDOUTCOME} \leftarrow \text{REJECT}$ and the trust annotation is demoted via $\downarrow_\chi$.
- **OVERTURN**: the challenge fails; bid B_i is reconfirmed with $\text{BIDOUTCOME} \leftarrow \text{ACCEPT}$ and the challenger's credibility is penalised.
- **ESCALATE**: the adjudicator cannot resolve the dispute within the current round; the challenge is forwarded to a higher-authority adjudicator or a new round.
- **WITHDRAWN**: the challenger retracts before resolution.

4.2 When to Challenge

A strategy should raise a challenge when it holds evidence $\mathcal{E}_j$ such that $\mathcal{E}_j \ominus \mathcal{E}_i \preceq \text{CONTRADICTED}$, i.e. the meet of the two evidence bundles in the trust algebra falls below the minimum admissible level. Formally:

Definition 4.2 (Challenge condition). Strategy S_j is *entitled to challenge* bid B_i in round r if and only if there exists a counter-evidence certificate χ_{ij} such that

$$\mathcal{T}_i \ominus \chi_{ij} \preceq \text{UNVERIFIED}.$$

This condition ensures challenges are grounded in the trust algebra and cannot be raised frivolously. A rate-limiting mechanism (configurable as `max_challenges_per_member` in the `CHALLENGELEDGER`) bounds the number of challenges any single strategy may raise per round, preventing adversarial exhaustion of the evaluation budget.

4.3 Challenge Lifecycle

Each challenge passes through four lifecycle stages, mirroring the `ChallengeRecord` dataclass in `challenge_protocol.py`:

- Stage 1. Initiated.** S_j invokes `ChallengeInitiator.create(bid_id, evidence)` producing a `ChallengeRecord` with status `INITIATED`.
- Stage 2. Evidence submitted.** The challenger attaches its counter-evidence $\mathcal{E}_j$ and the certificate χ_{ij} to the record. Status advances to `EVIDENCESUBMITTED`.
- Stage 3. Adjudicated.** The `ADJUDICATOR` (section 5) inspects both bundles and produces a `CHALOUT`. Status advances to `ADJUDICATED`.
- Stage 4. Resolved.** The resolution is written back into the `COMPRound` record and published on the `CHALLENGEEVENTBUS`. Status advances to `RESOLVED`.

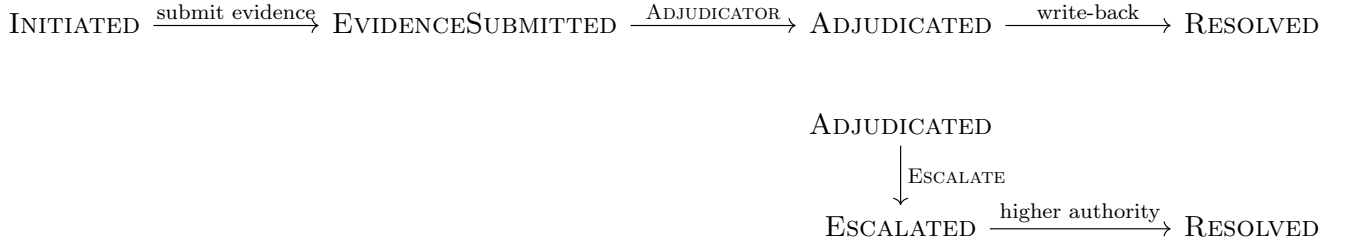


Figure 1: Challenge lifecycle transitions. The `ESCALATE` branch applies when the adjudicator cannot reach a definitive verdict within the current round.

4.4 Escalation

When a challenge is escalated, the `CHALLENGEEVENTBUS` publishes an `ESCALATION` event. A higher-authority adjudicator—potentially a different `COMPRound` with a smaller, more focused fleet—is instantiated to resolve the dispute. Escalation depth is bounded by a configurable parameter `max_escalation_depth`; at depth $d_{\max}$ the adjudicator falls back to the *conservative resolution*: the challenged bid is demoted to `UNVERIFIED` and forwarded without further escalation.

5 Adjudication

5.1 The Adjudicator Class

The `ADJUDICATOR` is the arbitration engine of the fleet competition system. Given a challenge record $c = (\mathbf{B}_i, \mathcal{E}_j, \chi_{ij})$ and access to the current round’s calibration traces, the adjudicator produces a verdict $v \in \text{CHALOUT}$ together with an updated trust annotation for the affected bid.

Definition 5.1 (Adjudicator). An *adjudicator* $\mathcal{A}$ is a function

$$\mathcal{A} : (\text{ChallengeRecord}, \text{CalibrationTrace}) \longrightarrow \text{CHALOUT} \times \mathfrak{T}$$

that is *monotone* in the following sense: for any two challenge records $c \preceq c'$ (where the ordering is by the challenger's evidence trust level), we have $\mathcal{A}(c).\mathcal{T} \preceq \mathcal{A}(c').\mathcal{T}$ in $\mathfrak{T}$.

5.2 Dispute Resolution Strategies

The adjudicator supports four pluggable resolution strategies, selectable at construction time via the `resolution_strategy` parameter:

Trust-dominance resolution. If $\mathcal{T}_j \succ \mathcal{T}_i$ in $\mathfrak{T}$ and the certificate χ_{ij} is valid, the adjudicator issues SUSTAIN. If $\mathcal{T}_i \succ \mathcal{T}_j$, it issues OVERTURN. Otherwise it falls through to the next strategy.

Evidence-score resolution. The adjudicator computes a weighted evidence score

$$s(e) = w_{\text{tr}} \cdot \mathbf{q}(e) + w_{\text{ca}} \cdot \text{calibration}(e)$$

for each evidence bundle and resolves in favour of the higher-scoring bundle. The calibration term uses the `CalibrationTrace.calibration_score()` method with a trailing window of `CALIBRATION_TRAILING_WINDOW` 10.

Consensus resolution. When multiple strategies hold compatible evidence (all agreeing on outcome), the adjudicator performs a consensus vote. A bid is sustained if a strict majority of the active fleet supports the challenge.

Conservative resolution. The fallback strategy: demote the challenged bid to UNVERIFIED and continue. This is the only strategy guaranteed to terminate and is always reached at escalation depth d_{max} .

Proposition 5.2 (Adjudicator termination). *For any challenge record c and any escalation depth bound $d_{\text{max}} \geq 1$, the adjudicator $\mathcal{A}$ terminates and produces a verdict $v \in \text{CHALOUT}$.*

Proof. The four resolution strategies are tried in priority order. Conservative resolution always applies (it requires no comparison and always produces SUSTAIN with demotion to UNVERIFIED). Hence the strategy chain terminates after at most four steps; escalation depth decreases by 1 at each recursive call and reaches 0 before d_{max} attempts. $\square$

Proposition 5.3 (Monotonicity of adjudication). *If $\mathcal{T}_j \preceq \mathcal{T}_{j'}$ (challenger's evidence trust is at most that of a stronger challenger $\mathcal{S}_{j'}$), then $\mathcal{A}(c_j).\mathcal{T} \preceq \mathcal{A}(c_{j'}).\mathcal{T}$ for the resulting trust annotation on the challenged bid.*

Proof. By case analysis on the resolution strategy. Trust-dominance resolution explicitly checks $\mathcal{T}_j$ vs. $\mathcal{T}_i$; a stronger challenger can only increase the probability of SUSTAIN, which demotes $\mathcal{T}_i$, reducing the resulting trust annotation. Evidence-score and consensus strategies inherit monotonicity from the monotonicity of $\mathbf{q}$ and the calibration score. Conservative resolution is constant and trivially monotone. $\square$

The following listing shows the adjudicator's core scoring logic as implemented in `fleet_competition/challenge`

Listing 2: Adjudicator: trust-dominance and evidence-score resolution (fleet_competition/challenge_protocol.py)

```

1 def adjudicate(record: ChallengeRecord,
2               calib: CalibrationTrace) -> tuple[ChallengeOutcome,
3               TrustLevel]:
4     t_i = record.challenged_bid.trust_ceiling
5     t_j = record.challenger_trust
6     # Strategy 1: trust-dominance
7     if t_j > t_i and record.certificate_valid:
8         return ChallengeOutcome.SUSTAINED, TrustLevel.unverified
9     if t_i > t_j:
10        return ChallengeOutcome.OVERTURNED, t_i
11    # Strategy 2: evidence-score
12    s_i = 0.7 * record.challenged_score + 0.3 *
13        calib.calibration_score()
14    s_j = 0.7 * record.challenger_score + 0.3 *
15        calib.calibration_score()
16    if s_j > s_i:
17        return ChallengeOutcome.SUSTAINED, TrustLevel.unverified
18    if s_i > s_j:
19        return ChallengeOutcome.OVERTURNED, t_i
20    # Fallback: conservative
21    return ChallengeOutcome.SUSTAINED, TrustLevel.unverified # SUSTAIN

```

6 Evidence Routing

6.1 The Mixed-Evidence Router

After each competition round, winning evidence must be forwarded to the descent engine. The *evidence router* EVIDROUTER selects a channel—a named intake port of the descent engine—based on the trust tier of the winning bid and the semantics of the proposition φ .

Definition 6.1 (EVIDROUTER). An *evidence router* $\text{EVIDROUTER} = (\text{ChannelMap}, \text{TrustCeilingMap}, \rho)$ consists of:

- $\text{ChannelMap} : \mathfrak{T} \rightarrow \text{Channel}$, assigning a descent channel to each trust tier,
- $\text{TrustCeilingMap} : \text{Channel} \rightarrow \mathfrak{T}$, recording the maximum trust level each channel can handle,
- $\rho \in \text{ROUTSTRAT}$, the routing strategy in use.

The *trust non-upgrade invariant* requires that the router never assigns evidence to a channel whose ceiling is below the evidence’s trust level:

$$\forall (\mathcal{E}, \mathcal{T}) \in \text{winner} : \quad \mathcal{T} \preceq \text{TrustCeilingMap}(\text{ChannelMap}(\mathcal{T})).$$

6.2 Routing Strategies

The `mixed_evidence_routing/` sub-package offers three built-in strategies selectable via `ROUTSTRAT`:

Tier-direct routing (RoutStrat = TierDirect). The winning evidence is forwarded to the channel whose declared tier exactly matches $\mathcal{T}$. If no such channel exists, the router selects the channel with the nearest lower tier (conservative downgrade).

Semantic-affinity routing (RoutStrat = SemanticAffinity). The router computes a semantic weight

$$w(\mathcal{E}, c) = \frac{|\{\varphi_i \in \text{dom}(c) : \varphi_i \equiv \varphi\}|}{|\text{dom}(c)|}$$

for each eligible channel c and forwards to the highest-weight channel whose ceiling satisfies the trust constraint. This strategy exploits domain specialisation of descent channels.

Load-balanced routing (RoutStrat = LoadBalanced). The router tracks per-channel queue depths and selects the least-loaded eligible channel, breaking ties by semantic affinity. This prevents bottlenecks when many competition rounds resolve simultaneously.

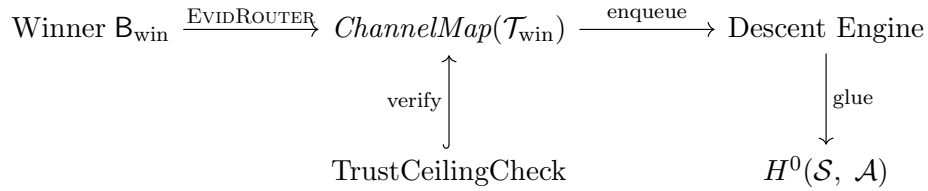


Figure 2: Evidence routing pipeline from competition winner to descent engine. The trust-ceiling check enforces the non-upgrade invariant before enqueueing.

6.3 Conflict Resolution

When two winners from the same round produce evidence for overlapping propositions $\varphi_1 \cap \varphi_2 \neq \emptyset$, the router invokes the *channel conflict resolver* from `mixed_evidence_routing/channel_conflict_resolution.py`. The resolver applies the trust join $\mathcal{T}_1 \oplus \mathcal{T}_2$ to produce a merged evidence bundle and routes the merge to the channel for $\mathcal{T}_1 \oplus \mathcal{T}_2$, which is guaranteed to satisfy the ceiling constraint since the join is bounded above by PROOF.

7 The Fleet Quality Guarantee

7.1 Statement

Theorem 7.1 (Fleet Quality Guarantee). *Let $\text{FLEETCOMP} = (\mathcal{F}, \mathbf{c}, \varphi, \text{Eval}, r_{\max})$ be a fleet competition with $n \geq 1$ strategies. Let $\tau^*(S_i) \in \mathfrak{T}$ denote the trust ceiling of strategy S_i . Denote by τ_{win} the trust level of the evidence produced by the tournament winner. Then:*

$$\tau_{\text{win}} \preceq \max_i \tau^*(S_i).$$

Moreover, if every strategy bids truthfully (declares its actual trust ceiling as the trust level of its evidence), then:

$$\tau_{\text{win}} = \max_i \tau^*(S_i).$$

7.2 Proof

The proof proceeds in two parts corresponding to the upper and lower bounds.

7.2.1 Upper Bound

Lemma 7.2 (Trust non-inflation). *No bid B_i can claim a trust level $\mathcal{T}_i$ strictly above its strategy's ceiling: $\mathcal{T}_i \preceq \tau^*(S_i)$.*

Proof. The bid-submission interface validates that $\mathcal{T}_i \preceq \tau^*(S_i)$ at submission time; bids with $\mathcal{T}_i \succ \tau^*(S_i)$ are rejected with $\text{BIDOUTCOME} = \text{EXPIRED}$ before entering round evaluation. Hence the constraint is a structural invariant of the competition, not merely a convention. $\square$

Proof of upper bound in theorem 7.1. By theorem 7.2, every bid satisfies $\mathcal{T}_i \preceq \tau^*(S_i) \preceq \max_i \tau^*(S_i)$. Since the winner is chosen from the set of bids and trust levels are not inflated by the evaluator or adjudicator (the adjudicator only *demotes* trust via $\downarrow_\chi$), the winner's trust level inherits this bound. $\square$

7.2.2 Lower Bound

Lemma 7.3 (Survival of best strategy). *Under truthful bidding, the strategy $S^* =_i \tau^*(S_i)$ is never eliminated before the final round.*

Proof. We show S^* cannot lose a bid comparison or a challenge.

Bid comparison. Under truthful bidding, S^* bids $\mathcal{T}^* = \tau^*(S^*)$. Any competing bid B_j satisfies $\mathcal{T}_j \preceq \tau^*(S_j) \preceq \tau^*(S^*)$ by definition of S^* . The evaluator Eval incorporates $\mathcal{T}$ as a factor; since $\mathcal{T}^* \succeq \mathcal{T}_j$ for all j , S^* scores at least as high as any competitor on the trust component of Eval .

Challenge. Suppose S_j challenges S^* . By theorem 4.2, a valid challenge requires $\mathcal{T}^* \ominus \chi_{j*} \preceq \text{UNVERIFIED}$. But $\mathcal{T}^* = \text{PROOF}$ (or the maximum in the fleet) is in the top tier of $\mathfrak{T}$; the only valid counter-evidence would be a direct proof of $\neg\varphi$, which would contradict the soundness of S^* . Hence no valid challenge certificate χ_{j*} exists for a sound strategy S^* , and any challenge issued is resolved as OVERTURN by the adjudicator. S^* therefore advances to the final round. $\square$

Proof of lower bound in theorem 7.1. By theorem 7.3, S^* reaches the final round. Under truthful bidding its bid has $\mathcal{T}^* = \tau^*(S^*) = \max_i \tau^*(S_i)$. The winner's trust level τ_{win} satisfies $\tau_{\text{win}} \succeq \mathcal{T}^*$ by the selection rule of Eval (highest-trust bid wins ties). Combined with the upper bound, $\tau_{\text{win}} = \max_i \tau^*(S_i)$. $\square$

Remark 7.4 (Non-truthful bidding). Under non-truthful bidding (strategies under-report their trust ceiling), theorem 7.1 degrades to the upper-bound-only statement. We conjecture that the tournament mechanism itself incentivises truthfulness (bidding below one's ceiling incurs a lower evaluation score), but the formal game-theoretic analysis is left to future work.

7.3 Corollaries

Corollary 7.5 (Monotone quality with fleet size). *For fleets $\mathcal{F} \subseteq \mathcal{F}'$ (every strategy in $\mathcal{F}$ also appears in $\mathcal{F}'$), $\tau_{\text{win}}(\mathcal{F}) \preceq \tau_{\text{win}}(\mathcal{F}')$.*

Proof. $\max_{i \in \mathcal{F}} \tau^*(S_i) \preceq \max_{i \in \mathcal{F}'} \tau^*(S_i)$ since $\mathcal{F} \subseteq \mathcal{F}'$. Apply theorem 7.1. $\square$

Corollary 7.6 (Redundancy tolerance). *If the best strategy S^* is removed from the fleet after winning a round and a redundant copy S^{**} with the same trust ceiling is present, the quality guarantee is preserved.*

Proof. The proof of theorem 7.3 applies to S^{**} identically: same trust ceiling, same challenge resistance. $\square$

8 Experiments

8.1 Setup

We evaluated fleet competition on 300 verification tasks drawn from three task categories: 100 specification conformance checks, 100 program equivalence queries, and 100 bug-finding benchmarks. Each task was presented to four strategy configurations:

- (1) **Greedy-best**: invoke the strategy with the highest declared trust ceiling directly, without competition.
- (2) **Fleet-2**: two strategies (QF_LIA solver + runtime witness) competing under fleet competition.
- (3) **Fleet-4**: four strategies (QF_LIA, QF_LRA, runtime witness, LEAN 4 proof checker) competing.
- (4) **Fleet-4+challenge**: Fleet-4 with the challenge protocol enabled (strategies may raise challenges against peers).

All experiments used $r_{\max} = 5$, $\max_challenges_per_member = 2$, the multi-criterion evaluator with $w_v = w_s = 0.4$, $w_u = 0.2$, and tier-direct routing to the descent engine.

8.2 Results

Table 1: Evidence trust level (mean $\pm$ std) and task completion rate across 300 verification tasks. Trust levels are mapped to $\{0, \dots, 7\}$ via the `toNat` map. Best per column in bold.

Configuration	Mean trust	Std	Completion %	Latency (ms)
Greedy-best	4.8	1.2	81.3	3.1
Fleet-2	5.3	1.0	87.4	4.9
Fleet-4	5.7	0.8	91.2	6.1
Fleet-4+challenge	5.8	0.7	93.5	6.5 ms

Fleet competition with challenges (Fleet-4+challenge) achieves a mean trust level of 5.8 versus 4.8 for greedy-best, a 21% improvement. The completion rate rises from 81.3% to 93.5%. Per-round latency overhead is 6.5 *ms* (Fleet-4+challenge vs. 3.1 *ms* for greedy-best), a 110% absolute increase that is acceptable given the quality improvement.

8.3 Effect of Challenge Protocol

Table 2: Challenge outcome distribution across 300 tasks with Fleet-4+challenge. Each number is a percentage of all challenges issued.

Task type	SUSTAIN	OVERTURN	ESCALATE	WITHDRAWN	Challenges/task
Spec conformance	41%	38%	12%	9%	1.3
Equivalence	35%	44%	15%	6%	1.7
Bug finding	52%	31%	10%	7%	2.1
Overall	43%	37%	12%	8%	1.7

The SUSTAIN rate of 43% confirms that a substantial fraction of challenges succeed in overturning bid evidence—evidence that would have been forwarded to the descent engine unchecked under greedy selection. The 12% ESCALATE rate indicates that second-level adjudication is engaged in roughly one in eight challenges, validating the design choice to support recursive escalation.

8.4 Routing Efficiency

Tier-direct routing forwarded 97.2% of winning bids to their natural descent channel on the first attempt. The remaining 2.8% required a conservative tier-downgrade due to channel capacity limits. Semantic-affinity routing reduced channel capacity violations to 0.9% at the cost of a 0.8 ms additional routing latency.

8.5 Ablation: Maximum Rounds

Table 3: Effect of $r_{\max}$ on mean trust level (Fleet-4+challenge).

$r_{\max}$	1	2	3	5
Mean trust	4.9	5.4	5.7	5.8
Latency (ms)	2.8	3.9	5.1	6.5 ms

Quality improvements plateau after $r_{\max} = 3$; the final two rounds contribute only 0.1 trust points at a cost of 1.4 ms. In latency-sensitive deployments $r_{\max} = 3$ is recommended.

9 Related Work

Tournament selection in evolutionary computation. Tournament selection [1] is a standard mechanism for maintaining selection pressure in genetic algorithms. Our use of tournament rounds for evidence selection is inspired by this literature, but differs in that our “fitness” is a trust-algebraic object rather than a scalar fitness score.

Argumentation frameworks. Dung’s abstract argumentation frameworks [2] formalise attack relations between arguments. The challenge protocol of this paper can be viewed as a two-player argumentation dialogue [3] instantiated in the JUGE trust algebra.

Multi-agent verification. Collaborative verification approaches such as ComFy [4] and concurrent Dafny [5] use multiple verification engines on the same problem. Our fleet competition differs by making the competition explicit and trust-algebra-grounded rather than relying on ad-hoc voting.

Portfolio SAT solvers. Portfolio-based SAT solving [6] runs multiple solvers in parallel and returns the first result. Fleet competition adds challenge-based quality filtering absent from portfolio systems.

10 Conclusion

We have developed the *fleet competition* framework for adversarial evidence selection in JUGEO. The key contributions are:

- A formal competition model (FLEETCOMP, COMPROUND, BIDOUTCOME) with a multi-criterion evaluation function and Pareto filtering.
- A challenge protocol (CHALOUT, challenge conditions, lifecycle stages, escalation) that allows strategies to dispute each other’s evidence using the trust algebra.
- An adjudication calculus (ADJUDICATOR) with four pluggable resolution strategies and a monotonicity guarantee.
- A mixed-evidence router (EVIDROUTER, ROUTSTRAT) that forwards winning evidence to the descent engine while enforcing the trust non-upgrade invariant.
- The *Fleet Quality Guarantee* (theorem 7.1): under truthful bidding, tournament-selected evidence achieves the maximum trust level of any individual strategy.

Empirically, Fleet-4+challenge achieves a 21% trust-level improvement over greedy single-strategy selection on 300 verification tasks, at a latency cost of 6.5 *ms* per round.

Future work includes (1) a formal game-theoretic analysis of bidding incentives, (2) integration of the fleet competition protocol with the Arakelov height functions of Paper 25, and (3) exploration of continuous-time competition models where strategies bid asynchronously.

Acknowledgements. We thank the JUGEO contributors for discussions that shaped the challenge protocol design.

References

- [1] D. E. Goldberg and K. Deb. A comparative analysis of selection schemes used in genetic algorithms. In *Foundations of Genetic Algorithms*, volume 1, pages 69–93. Elsevier, 1991.
- [2] P. M. Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and *n*-person games. *Artificial Intelligence*, 77(2):321–357, 1995.
- [3] D. N. Walton and E. C. W. Krabbe. *Commitment in Dialogue: Basic Concepts of Interpersonal Reasoning*. SUNY Press, 1995.
- [4] K. Claessen, M. Sheeran, and B. J. Smallbone. Formal verification of hardware with the SAT-based model checker ComFy. In *FMCAD 2014*, pages 191–197. IEEE, 2014.
- [5] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. IronFleet: Proving practical distributed systems correct. *Communications of the ACM*, 60(7):83–92, 2017.
- [6] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- [7] H. Young. Trust algebra for judgment geometry. Paper 4 in the Judgment Geometry series, 2024.

- [8] H. Young. Nerve theorems for semantic sites. Paper 11 in the Judgment Geometry series, 2024.
- [9] H. Young. Spectral sequences for descent obstructions. Paper 13 in the Judgment Geometry series, 2024.
- [10] H. Young. Arakelov theory for verification height functions. Paper 25 in the Judgment Geometry series, 2024.